

The most effective prompts are those that are clear, concise, specific, and include examples of exactly what a response should look like. In this chapter, we will cover several strategies and tactics to get the most effective responses from the Command family of models. We will cover formatting and delimiters, context, using examples, structured output, do vs. do not do, length control, begin the completion yourself, and task splitting. We will highlight best practices as a user crafting prompts in the Cohere playground, as well as through the API.

Formatting and Delimiters

A clear, concise, and specific prompt can be more effective for an LLM with careful formatting. Instructions should be placed at the beginning of the prompt, and different types of information, such as instructions, context, and resources, should be delimited with an explanatory header. Headers can be made more clear by prepending them with `##`.

For example:

```
## Instructions
Summarize the text below.

## Input Text
{input_text}
```

Then use the Chat API to send a message to the model:

```
PYTHON

1  import cohere
2
3  co = cohere.ClientV2(api_key="<YOUR API KEY>")
4
5  message = ""
6  ## Instructions
7  Summarize the text below.
8
9  ## Input Text
10 {input_text}
11 ""
12
13 # get model response
14 response = co.chat(
15     messages=[{"role": "user", "content": message}],
16     model="command-a-plus-05-2026",
17     temperature=0.3,
18 )
```

Context

The previous prompt has concise instructions that begin the prompt (“summarize the text”) and is formatted clearly, where the instructions and resources are separated with delimiters. However, it lacks context that the LLM could use to produce a better-quality summary for the desired output. Including information about the input text could improve the prompt.

```
## Instructions
Below there is a long form news article discussing the 1972 Canada-USSR Summit, an eight-game ice hockey series between the Soviet Union and Canada, held in 1972. Please summarize the salient points of the text and do so in a flowing high quality text. Use bullet points where appropriate.

## News Article
{news_article}
```

While embedding a news article directly in a prompt works well, Cohere grounded generation is directly available through the Chat API which can result in a much improved completion. Grounded completion focuses on generating accurate and relevant responses by avoiding preambles, or having to include documents directly in the message. The benefits include:

- Less incorrect information.
- More directly useful responses.
- Responses with precise citations for source tracing.

For this method, we recommend providing documents through the `documents` parameter. Our models process conversations and document snippets (100-400 word chunks in key-value pairs) as input, and you have the option of including a system message.

For the example above, we can chunk a news article into different sections and attach them via the `documents` field in the `user` message. The Chat API will then provide us not only with the completion but also citations that ground information from the documents. See the following:

PYTHON

```
1 # Sections from the original news article
2 document_chunked = [
3     {
4         "data": {
5             "text": "Equipment rental in North America is predicted to "normalize" going into 2024, according to a new report from the Equipment Leasing and Finance Association (ELFA). The report states that the equipment rental industry is expected to grow at a steady pace over the next few years, with a projected CAGR of 3.5% from 2023 to 2028. This growth is driven by a combination of factors, including the increasing demand for specialized equipment in various industries, the expansion of the construction sector, and the growing adoption of new technologies in the rental space. The report also highlights the importance of maintaining a strong relationship with customers and providing excellent service to ensure long-term success in the market."
6         }
7     },
8     {
9         "data": {
10            "text": "\"Rental is going back to 'normal,' but normal means that strategy matters again - growth is expected to be steady, not explosive. The industry is seeing a return to normalcy after a period of rapid growth, but this means that companies need to focus on their core competencies and develop a clear strategy for the future. The report also notes that the industry is facing challenges such as inflation and supply chain issues, which could impact growth in the near term. However, the long-term outlook remains positive, with the industry expected to continue to grow and evolve over the next decade."
11        }
12    },
13    {
14        "data": {
15            "text": "\"Everybody was breaking records, from the national rental chains to the smallest regional players. The industry was experiencing a period of rapid growth, with many companies reporting record sales and profits. This was due to a combination of factors, including the increasing demand for equipment rental services, the expansion of the construction sector, and the growing adoption of new technologies in the rental space. The report also notes that the industry is facing challenges such as inflation and supply chain issues, which could impact growth in the near term. However, the long-term outlook remains positive, with the industry expected to continue to grow and evolve over the next decade."
16        }
17    },
18 ]
19
20 # Add a system message for additional context
21 system_message = """## Task and Context
22 You will receive a series of text fragments from a document that are presented in chronological order. A
23
24 # Call the model
25 message = f"Summarize this text in one sentence."
26
27 response = co.chat(
28     model="command-a-plus-05-2026",
29     documents=document_chunked,
30     messages=[
31         {"role": "system", "content": system_message},
32         {"role": "user", "content": message},
33     ],
34 )
35
36 response_text = response.message.content[0].text
37
```

```
38 print(response_text)
```

The model returns a concise summary as instructed:

```
Josh Nickell, vice president of the American Rental Association, predicts t
```

But importantly, it also returns citations that ground the completion in the included `documents`. The citations are returned in `response.message.citations` as a list of JSON dictionaries:

```
[Citation(start=0,
          end=12,
          text='Josh Nickell',
          sources=[DocumentSource(type='document', id='doc:0', document={'id'
Citation(start=118,
          end=129,
          text='"normalize"',
          sources=[DocumentSource(type='document', id='doc:0', document={'id'
Citation(start=133, ...
```

These can easily be rendered into the text to show the source of each piece of information. The following Python function adds the returned citations to the returned completion.

PYTHON

```
1  # Function to insert inline citations into the text
2  def insert_inline_citations(text, citations):
3      sorted_citations = sorted(
4          citations, key=lambda c: c.start, reverse=True
5      )
6
7      for citation in sorted_citations:
8          source_ids = [
9              source.id.split(":")[-1] for source in citation.sources
10             ]
11             citation_text = f"[{'.'.join(source_ids)}]"
12             text = (
13                 text[: citation.end]
14                 + citation_text
15                 + text[citation.end :]
16             )
17
18     return text
19
20
21 # Function to list source documents
22 def list_sources(citations):
23     unique_sources = {}
24     for citation in citations:
25         for source in citation.sources:
26             source_id = source.id.split(":")[-1]
27             if source_id not in unique_sources:
28                 unique_sources[source_id] = source.document
29
30     footnotes = []
31     for source_id, document in sorted(unique_sources.items()):
32         footnote = f"[{source_id}] "
33         for key, value in document.items():
34             footnote += f"{key}: {value}, "
35         footnotes.append(footnote.rstrip(", "))
36
37     return "\n".join(footnotes)
38
39
40 # Use the functions
41 cited_text = insert_inline_citations(
42     response.message.content[0].text, response.message.citations
43 )
```

```

44
45 # Print the result with inline citations
46 print(cited_text)
47
48 # Print source documents
49 if response.message.citations:
50     print("\nSource documents:")
51     print(list_sources(response.message.citations))

```

Sample output

```

Josh Nickell[0], vice president of the American Rental Association[0], predicted that
the rental industry is going to see a significant increase in demand over the next
few years.

Source documents:
[0] id: doc:0, text: Equipment rental in North America is predicted to “normalize”
[1] id: doc:1, text: “Rental is going back to ‘normal,’ but normal means that the
[2] id: doc:2, text: “Everybody was breaking records, from the national rental

```

Incorporating Example Outputs

LLMs respond well when they have specific examples to work from. For example, instead of asking for the salient points of the text and using bullet points “where appropriate”, give an example of what the output should look like.

```

## Instructions
Below there is a long form news article discussing the 1972 Canada-USSR Summit.

## Example Output
High level summary: <summary>
3 important events related to the series:
* <important event 1>
* <important event 2>
* <important event 3>

## News Article
{news_article}

```

Structured Output

In addition to examples, asking the model for structured output with a clear and demonstrated output format can help constrain the output to match desired requirements. JSON works particularly well with the Command R models.

```

Output the summary in the following JSON format:
{
  "short_summary": "<include a short summary of the text here>",
  "most_important_events": [
    "<one important event>",
    "<another important event>",
    "<another important event>"
  ]
}

```

Do vs. Do Not Do

Be explicit in **exactly** what you want the model to do. Be as assertive as possible and avoid language that could be considered vague. To encourage abstract summarization, do not write something like “avoid extracting full sentences from the input text,” and instead do the following:

```

## Instructions
Below there is a long form news article discussing the 1972 Canada-USSR Summit.

```

```
Paraphrase the content into re-written, easily digestible sentences. Do not
```

```
## News Article  
{news_article}
```

Length Control

Command A models excel at length control. Use this to your advantage by being explicit about the desired length of completion. Different units of length work well, including paragraphs (“give a summary in two paragraphs”); sentences (“make the response between 3 and 5 sentences long”); and words (“the completion should be at least 100 and no more than 200 words long”).

```
...  
The output summary should be at least 250 words and no more than 300 words
```

Begin the Completion Yourself

LLMs can easily be constrained by beginning the completion as part of the input prompt. For example, if it is very important that the output is HTML code and that it must be a well-formed HTML document, you can show the model how the completion should begin, and it will tend to follow suit.

```
...  
Please generate the response in a well-formed HTML document. The completion  
follows:  
  
<!DOCTYPE html>  
<html>
```

Task Splitting

Finally, task splitting should be used when the requested task is complex and can be broken down into sub-tasks. Doing this for the model can help guide it to the best possible answer. Instead of asking for a summary of the most important sentence in the most important paragraph in the input, break it down piece by piece in the prompt:

```
## Instructions  
Using the included text below, perform the following steps:  
  
1. Read through the entire text carefully  
2. Extract the most important paragraph  
3. From the paragraph extracted in step 2, extract the most important sentence  
4. Summarize the sentence extracted in step 3 and make it between 30 and 50 words  
5. Only return the result of step 4 in your response.
```

In the next chapter, we will discuss more advanced prompt engineering techniques, including few-shot prompting and chain-of-thought.